



第四章 秘密共享和茫然传输

目 录

「01」

知识点一：基本概念

「02」

知识点二：Shamir方案

「03」

知识点三：秘密共享应用实践

「04」

知识点四：茫然传输

1. 定义

- 秘密共享 (Secret Sharing) 是一种**将秘密分割存储的密码技术**，目的是阻止秘密过于集中，以达到分散风险和容忍入侵的目的，是信息安全和数据保密中的重要手段。
- 目前，**秘密共享已成为一种重要密码学工具**，在诸多多方安全计算**协议中被使用**，例如拜占庭协议、多方隐私集合求交协议、阈值密码学等。

1. 定义

定义 令 D 为秘密值所在域，令 D_1 为秘密份额所在域。令 $Shr: D \rightarrow D_1^n$ 为**秘密分配**算法（可能是随机性算法）， $Rec: D_1^k \rightarrow D$ 为**秘密重构**算法，其中 D_1^n 表示 n 个秘密份额、 D_1^k 表示 k 个秘密份额。

(t, n) – **秘密分享方案包含一对算法 (Shr, Rec) ，满足下述两个性质：**

- **正确性。** 令 $(s_1, s_2, \dots, s_n) = Shr(s)$ ，则：

$$Pr[\forall k \geq t, Rec(s_{i_1}, \dots, s_{i_k}) = s] = 1$$

- **完美隐私性。** 任意包含少于 t 个秘密份额的集合都不会在信息论层面上泄漏与秘密值相关的任何信息。

我们在多数情况下使用的都是 (n, n) – 秘密分享方案，即拥有全部 n 个秘密份额是重建出秘密值的充分必要条件。

2. 分类

- 基于秘密分配和运算的形式，将秘密共享分为基于位运算的加性秘密共享和基于线性代数的线性秘密共享等。
- 依据秘密重构的条件或者秘密分享的份额数量等，又可以将秘密共享分为如下类别：
 - 门限秘密共享**：任意大于等于阈值的参与方集合可重构出秘密。
 - 多重秘密共享**：参与方的子秘密可以多次使用，分别恢复多个共享秘密
 - 多秘密共享**：一次共享，共享多个秘密，且子秘密可以重复使用
 - 可验证秘密共享**：可通过公共变量验证自己子秘密的正确性
 - 动态秘密共享**：允许添加或删除参与方，定期或不定期更新参与方的子秘密，还允许在不同的时间恢复不同的秘密。

3. 门限秘密共享

在1979年, Shamir提出了门限秘密共享算法。

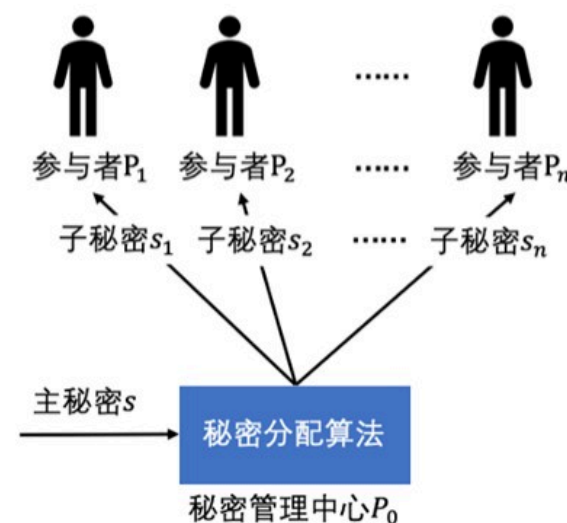
- 设 t 和 n 为两个正整数, 且 $t \leq n$, n 个需要共享秘密的参与者集合为 $P = \{P_1, \dots, P_n\}$ 。

一个 (t, n) 门限秘密共享体制是指: 假设 $P_1, \dots, P_n$ 要共享同一个秘密 s , 将 s 称为主秘密, 至少 t 个参与者才可以共同恢复主秘密 s 。

- 有一个秘密管理中心 P_0 来负责对 s 进行管理和分配, P_0 掌握有秘密分配算法和秘密重构算法, 这两个算法均满足重构要求和安全性要求。

3. 门限秘密共享

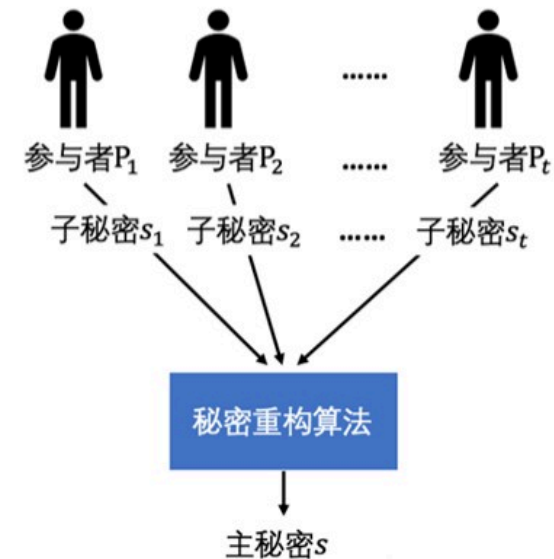
秘密分配。秘密管理中心 P_0 首先通过将主秘密 s 输入秘密分配算法，生成 n 个值，分别为 $s_1, \dots, s_n$ ，称 $s_1, \dots, s_n$ 为子秘密。然后秘密管理中心 P_0 分别将秘密分配算法产生的子秘密 $s_1, \dots, s_n$ 通过 P_0 与 P_i 之间的安全通信信道秘密地传送给参与者 P_i ，参与者 P_i 不得向任何人泄露自己所收到的子秘密 s_i 。



3. 门限秘密共享

秘密重构。门限值 t 指的是任意大于或等于 t 个参与者 P_i ，将各自掌握的子秘密 s_i 进行共享，**任意的一个参与者 P_i 在获得其余 $t - 1$ 个参与者所掌握的子秘密后，都可独立地通过秘密重构算法恢复出主秘密 s 。**

安全性要求任意攻击者通过收买等手段获取了少于 t 个的子秘密，或者任意少于 t 个参与者串通都无法恢复出主秘密 s ，也无法得到主秘密 s 的信息。



目 录

『01』

知识点一：基本概念

『02』

知识点二：Shamir方案

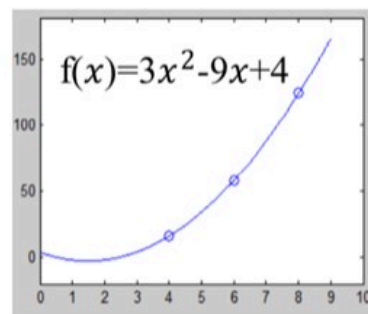
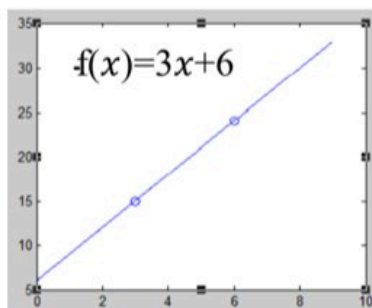
『03』

知识点三：秘密共享应用实践

『04』

知识点四：茫然传输

1. Shamir方案



Shamir于1979年基于多项式插值算法设计了Shamir(t, n)门限秘密共享体制，思路：

- 平面上不同的两个点唯一的确定平面上的一条直线，即一次多项式。
- 平面上不同的三个点唯一的确定平面上一个二次多项式。
- 一般的，设 $\{(x_1, y_1), \dots, (x_k, y_k)\}$ 是平面上 k 个不同的点构成的点集，那么在平面上存在**唯一**的 $k - 1$ 次多项式 $f(x) = a_0 + a_1x + \dots + a_{k-1}x^{k-1}$ 通过这 k 个点。
- 若把秘密 s 取做 $f(0)$ ， n 个份额取做 $f(i)$ ($i = 1, \dots, n$)，那么利用其中任意 k 个份额就可以重构 $f(x)$ ，从而得到秘密 $s = f(0)$ 。

1. Shamir方案

(1) 秘密分配算法

假设 $\mathbb{F}_q$ 为 q 元有限域, q 是素数且 $q > n$ 。 $P = \{P_1, \dots, P_n\}$ 是参与者集合, P 共享主秘密 s , $s \in \mathbb{F}_q$, 秘密管理中心 P_0 按如下步骤对主秘密 s 进行分配 (略去了模 q 操作) :

1. P_0 秘密地在有限域 $\mathbb{F}_q$ 中随机选取 $t - 1$ 个元素, 记为 $a_1, \dots, a_{t-1}$, 并取以 x 为变元的多项式 $f(x) = a_{t-1}x^{t-1} + \dots + a_1x^1 + \textcolor{red}{s} = s + \sum_{i=1}^{t-1} a_i x^i$ 。
2. 对于 $1 \leq i \leq n$, P_0 秘密计算 $\textcolor{red}{y}_i = \textcolor{red}{f}(i)$ 。
3. 对于 $1 \leq i \leq n$, P_0 通过安全信道秘密地将 (i, y_i) 分配给 P_i 。

1. Shamir方案

(2) 秘密重构算法

通常有两类方法：解方程法和多项式插值法。

算法一：解方程法

解方程法比较通俗，即 t 个方程可以确定 t 个未知数，而这 t 个未知数即为包括主秘密 s 在内的多项式 $f(x)$ 的各项系数。

如果参与者 $P_1, \dots, P_t$ 掌握了子秘密 $f(1), \dots, f(t)$ ，解方程：

$$a_{t-1}1^{t-1} + \dots + a_11^1 + s = f(1)$$

$$a_{t-1}2^{t-1} + \dots + a_12^1 + s = f(2)$$

$$\vdots$$

$$a_{t-1}t^{t-1} + \dots + a_1t^1 + s = f(t)$$

即可求解出系数 $a_{t-1}, \dots, a_1, s$ 。

1. Shamir方案

算法二：多项式插值法

假设 t 个子秘密分别为 (x_i, y_i) ，其中 $y_i = f(x_i)$, $i = 1, \dots, t$ 且 $i \neq j$ 时 $x_i \neq x_j$ ，参与者 $P_1, \dots, P_t$ 计算：

$$\begin{aligned} h(x) &= y_1 \frac{(x - x_2)(x - x_3) \cdots (x - x_t)}{(x_1 - x_2)(x_1 - x_3) \cdots (x_1 - x_t)} + y_2 \frac{(x - x_1)(x - x_3) \cdots (x - x_t)}{(x_2 - x_1)(x_2 - x_3) \cdots (x_2 - x_t)} + \cdots \\ &\quad + y_t \frac{(x - x_1)(x - x_2) \cdots (x - x_{t-1})}{(x_t - x_1)(x_t - x_2) \cdots (x_t - x_{t-1})} \end{aligned}$$

显然， $h(x)$ 是一个 $t - 1$ 次的多项式，且因为 $i \neq j$ 时 $x_i \neq x_j$ ，每个加式的分母均不为零，因此对于 $i = 1, \dots, t$ ， $y_i = h(x_i) = f(x_i)$ 。又根据多项式的性质，如果存在两个最高次均为 $t - 1$ 次的多项式，这两个多项式在 t 个互不相同的点所取的值均相同，那么这两个多项式相同，即 $h(x) = f(x)$ 。因此，参与者 P_i 通过分享各自秘密，共同计算 $h(0) = f(0) = s$ ，即可恢复主秘密 s 。

1. Shamir方案

(3) 举例说明: (3,5)门限方案

设 $t = 3$, $n = 5$, $q = 19$, $s = 11$ 。随机选取系数 $a_1 = 2$, $a_2 = 7$, 则:

$$f(x) = 7x^2 + 2x + 11 \bmod 19$$

计算可知, $f(1) = 1$, $f(2) = 5$, $f(3) = 4$, $f(4) = 17$, $f(5) = 6$ 。若已知 $f(2)$, $f(3)$, $f(5)$:

- 使用解方程法解如下方程组:

$$(a_2 \times 2^2 + a_1 \times 2 + s) \bmod 19 = f(2) = 5$$

$$(a_2 \times 3^2 + a_1 \times 3 + s) \bmod 19 = f(3) = 4$$

$$(a_2 \times 5^2 + a_1 \times 5 + s) \bmod 19 = f(5) = 6$$

可解得 $a_1 = 2$, $a_2 = 7$, $s = 11$ 。

- 使用多项式插值法, 由拉格朗日插值公式可知:

$$f(x) = 5 \frac{(x-3)(x-5)}{(2-3)(2-5)} + 4 \frac{(x-2)(x-5)}{(3-2)(3-5)} + 6 \frac{(x-2)(x-3)}{(5-2)(5-3)} = 7x^2 + 2x + 11$$

故 $s = f(0) = 11$ 。

2. 插值原理

○ 拉格朗日插值定义

拉格朗日插值公式指的是在节点上给出节点基函数，然后做基函数的线性组合，组合系数为节点函数值的一种插值多项式。

已知 $f(x)$ 在区间 $[a, b]$ 上一组 $n + 1$ 个不同点 $a \leq x_0, x_1, \dots, x_n \leq b$ 上的函数值 $y_i = f(x_i), (i = 0, 1, 2, \dots, n)$,

构造满足插值条件 $L_n(x_i) = y_i (i = 0, 1, 2, \dots, n)$ 的次数不超过 n 次的多项式 $L_n(x) = \sum_{i=0}^n \varphi_i(x) f(x_i) \approx f(x)$,

其中 $\varphi_i(x) (i = 0, 1, 2, \dots, n)$ 是次数不超过 n 次的多项式。

2. 插值原理

○ 两点拉格朗日插值

设已知两个不同节点 x_0, x_1 上的函数值 $f(x_0) = y_0, f(x_1) = y_1$,构造满足插值条件 $L_1(x_i) = y_i$ ($i = 0, 1$)的次数不超过一次的多项式

$$L_1(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1,$$

其中 $\varphi_i(x)$ ($i = 0, 1$)是次数不超过1次的多项式。

实际上就是构造一次的多项式 $\varphi_0(x)$ 和 $\varphi_1(x)$ 并满足:

$$L_1(x_0) = \varphi_0(x_0)y_0 + \varphi_1(x_0)y_1 = y_0$$

$$L_1(x_1) = \varphi_0(x_1)y_0 + \varphi_1(x_1)y_1 = y_1$$

2. 插值原理

○ 两点拉格朗日插值

令 $\varphi_0(x)$ 和 $\varphi_1(x)$ 满足如下条件显然可以使上述两式成立:

	x_0	x_1
$\varphi_0(x)$	1	0
$\varphi_1(x)$	0	1

由于 $\varphi_0(x)$ 和 $\varphi_1(x)$ 均为一次多项式, 因此可求得:

$$\varphi_0(x) = \frac{x - x_1}{x_0 - x_1}, \varphi_1(x) = \frac{x - x_0}{x_1 - x_0}$$

求解过程: 因为是1次, $\varphi_0(x) = kx + b$, 因为 $\varphi_0(x_0) = 1, \varphi_0(x_1) = 0$, 因此可以得到上述

等式。进而有: $L_1(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1 = \frac{x-x_1}{x_0-x_1}y_0 + \frac{x-x_0}{x_1-x_0}y_1$

这就是两点拉格朗日插值公式, 用它来作为函数 $y = f(x)$ 的近似。

显然 $L_1(x)$ 是次数不超过一次的多项式, 且满足:

$$L_1(x_0) = \varphi_0(x_0)y_0 + \varphi_1(x_0)y_1 = 1 \cdot y_0 + 0 \cdot y_1 = y_0$$

$$L_1(x_1) = \varphi_0(x_1)y_0 + \varphi_1(x_1)y_1 = 0 \cdot y_0 + 1 \cdot y_1 = y_1$$

2. 插值原理

○ 多点拉格朗日插值

我们可以用同样的方法建立多点的拉格朗日插值公式。

设已知 $n + 1$ 个不同节点 $x_0, x_1, \dots, x_n$ 上的函数值 $f(x_0) = y_0, f(x_1) = y_1, f(x_2) = y_2, \dots, f(x_n) = y_n$,

构造满足插值条件 $L_n(x_i) = y_i$ ($i = 0, 1, 2, \dots, n$)的次数不超过 n 次多项式

$$L_n(x) = \varphi_0(x)y_0 + \varphi_1(x)y_1 + \dots + \varphi_n(x)y_n$$

其中 $\varphi_i(x)$ ($i = 0, 1, \dots, n$)是次数为 n 的多项式。

2. 插值原理

○ 多点拉格朗日插值

同前述方法，令 $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$ 满足如下条件：

	x_0	x_1	$\dots$	x_n
$\varphi_0(x)$	1	0	$\dots$	0
$\varphi_1(x)$	0	1	$\dots$	0
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$\varphi_n(x)$	0	0	$\dots$	1

可求得：

$$\varphi_i(x) = \frac{(x - x_0)(x - x_1) \cdots (x - x_{i-1})(x - x_{i+1}) \cdots (x - x_n)}{(x_i - x_0)(x_i - x_1) \cdots (x_i - x_{i-1})(x_i - x_{i+1}) \cdots (x_i - x_n)} = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

$$L_n(x) = \sum_{i=0}^n \varphi_i(x) y_i = \sum_{i=0}^n \left(\prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} \right) y_i$$

这就是 $n + 1$ 点拉格朗日插值公式。

3. 同态特性

下述以**两个秘密的加乘**为例进行讲解。

假设输入的秘密 a 和秘密 b 已经通过随机多项式 $f_a(x)$ 和 $f_b(x)$ 利用Shamir门限体制共享给了各个参与者, $f_a(x) = m_{t-1}x^{t-1} + \dots + m_1x^1 + a$, $f_b(x) = n_{t-1}x^{t-1} + \dots + n_1x^1 + b$, 其中 $m_{t-1}, \dots, m_1$ 、 $n_{t-1}, \dots, n_1$ 是随机的多项式系数。

参与者 P_i 掌握输入 a 的子秘密 a_i 和输入 b 的子秘密 b_i 。

3. 同态特性

○ 加法同态

每个参与者 P_i 独立计算 $c_i = a_i + b_i, 1 \leq i \leq n$ 。 $c_1, \dots, c_n$ 即为 $a+b$ 通过随机多项式共享后的结果，通过多项式插值法或者解方程即可恢复秘密 s 。

子秘密可以直接相加，是因为对于 $c_i = a_i + b_i = f_a(i) + f_b(i) = (m_{t-1} + n_{t-1})i^{t-1} + \dots + (m_1 + n_1)i + a + b$ ，多项式的次数并没有发生变化，新的多项式 $f_a(x) + f_b(x)$ 的最高次数依旧为 $t-1$ 。

因此， t 个参与者共享他们计算出来的 c_i ，即可根据 t 个方程解 t 个未知数，解出 $a+b$ 。

当然，参与者也可以使用拉格朗日插值法求解出 $a+b$ 。

注意，这里不需要共享每个参与者掌握的 a_i 和 b_i 。

3. 同态特性

○ 乘法同态

Shamir秘密共享具有受限的乘法同态性质，其受限来自于2个 $t - 1$ 次多项式相乘其结果是 $2(t - 1)$ 次多项式，根据拉格朗日插值定理需要 $2(t - 1) + 1$ 个秘密份额才能恢复出多项式。如果 $2(t - 1) + 1 > n$ ，那将会没有足够的秘密份额进行插值。

- 因此，要使Shamir秘密共享满足乘法同态的性质，需要增加其他的计算机制。比如：离散对数拥有如下性质： $\log_g ab = \log_g a + \log_g b$,
- 对秘密 a 和秘密 b 的对数 $\log_g a$ 和 $\log_g b$ 进行Shamir秘密分享
- 通过加法同态，可以得到秘密 a 、秘密 b 的对数 $\log_g a$ 、 $\log_g b$ 的和 $\log_g a + \log_g b = \log_g ab$ ，即秘密 a 和秘密 b 的乘积的对数，进一步做指数运算就可以得到秘密的积 ab 。

单选题 1分

有关Shamir秘密共享，说法错误的是

- ☐ A 可以基于解方程法来实现秘密重构
- ☐ B 可以基于多项式插值法来实现秘密重构
- ☐ C 具有不受限制的加法同态性质
- ☒ D 具有无限的乘法同态性质

目 录

『01』

知识点一：基本概念

『02』

知识点二：Shamir方案

『03』

知识点三：秘密共享应用实践

『04』

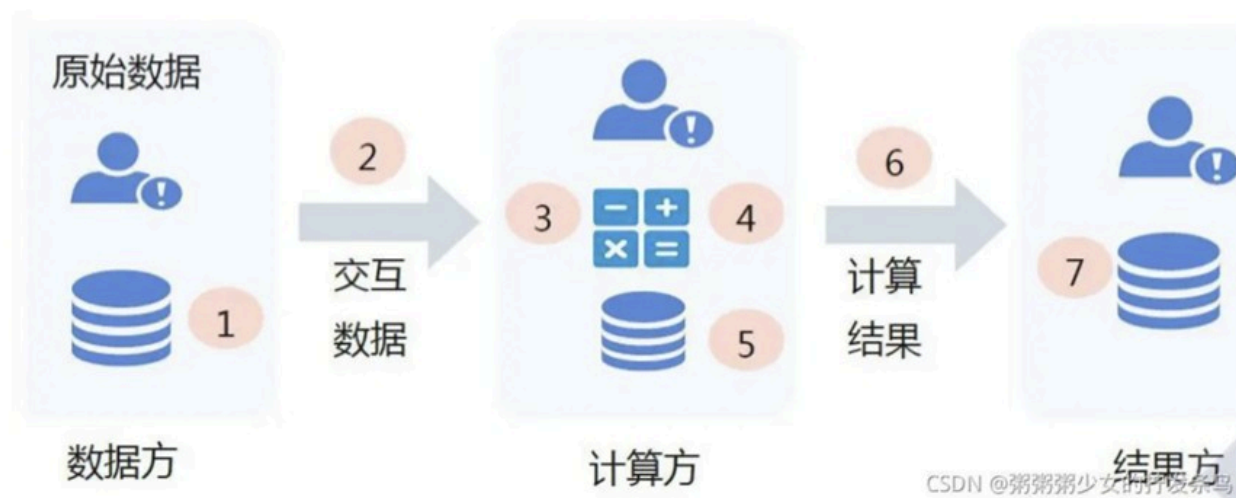
知识点四：茫然传输

引子

秘密共享的作用远不止允许多个秘密拥有方完成秘密的恢复，多数秘密共享方案都具有同态重构的特性，可以用来构建多方安全计算协议，即：**参与者在本地拥有的多个秘密份额上进行加法和乘法计算，进而可以通过秘密重构的方法完成运算结果的恢复，就是明文消息加或者乘之后的结果。**

1. 多方计算示意

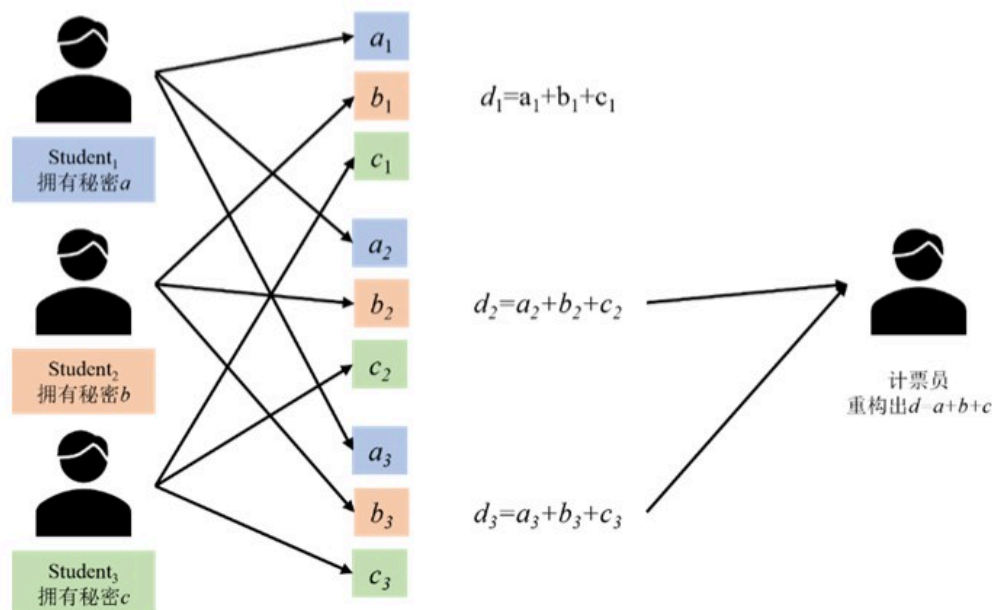
安全多方计算场景通常有三类角色：**数据方**、**计算方**和**结果方**。其中，数据方拥有隐私的输入数据；计算方提供安全计算的能力，但不能知道数据方的隐私数据，也不能通过计算过程获得有用的信息；结果方允许知道运算的结果，但不能知道隐私的输入信息。



在实际应用中，一些身份允许共有，只要不违背协议的安全要求即可。比如，班级里 n 个同学进行投票，他们的电脑可以作为计算节点，而且一位同学被设定为计票员，也就是该同学既可以投票又可以计票，会同时出现数据方、计算方和结果方的三重身份。

2. 投票统计示例

实验3.2 假设有三个同学需要对班里的优秀干部Alice、Bob、Charles、Douglas进行投票，最后统计各个班干部获得的票数。这个时候就可以利用Shamir秘密共享将各个投票方的投票分享出去并进行隐私求和计算。



对一个选手计票，每人拥有秘密0或者1

- 三个投票同学做计算方
- (2,3) 门限共享
- 三个同学之外选一个计票员
- 投票方共享3份秘密
- 投票方各自将3份秘密相加
- 计票员任意选2个进行重构

目 录

『01』

知识点一：基本概念

『02』

知识点二：Shamir方案

『03』

知识点三：秘密共享应用实践

『04』

知识点四：茫然传输

1. 基本概念

- 茫然传输(Oblivious Transfer, OT)是安全多方计算(Multi-Party Computation, MPC)协议的重要构造模块。
- 2选1-OT的标准定义涉及两个参与方：持有两个秘密值 x_0, x_1 的发送方 S ，持有一个选择比特 $b \in \{0, 1\}$ 的接收方 R 。OT允许 R 得到 x_b ，但其无法得到与“另一个”秘密值 x_{1-b} 相关的任何信息。与此同时， S 无法得到任何信息。
- 举例：Alice这儿有两个产品的折扣码，Bob想获得其中一个。但Bob又比较注重隐私，不想让Alice知道他选择了哪一个。这时候他们就可以通过OT来完成交易。

1. 基本概念

OT的形式化定义如下所述。

定义：2选1-OT是一个密码学协议，此协议可以安全地实现下图所示的功能函数 F^{OT} 。

参数：

- 两个参与方：发送方 S 和接收方 R ， S 拥有两个秘密值 $x_0, x_1 \in \{0,1\}^n$ ， R 拥有一个选择比特 $b \in \{0,1\}$ 。

功能函数：

1. R 将 b 发送给 F^{OT} ， S 将 x_0, x_1 发送给 F^{OT} 。
2. R 收到 x_b ， S 收到 $\perp$ 。

还有很多OT变种协议，一种很容易想到的就是 k 选1-OT，其中 S 拥有 k 个秘密值， R 拥有 $[0, \dots, k-1]$ 中的一个选择项。

2. 基础构造

基于公钥的OT协议。我们先讲解半诚实攻击模型下基于公钥的 OT 协议。下图给出了协议的构造方法。此协议的原理非常简单。

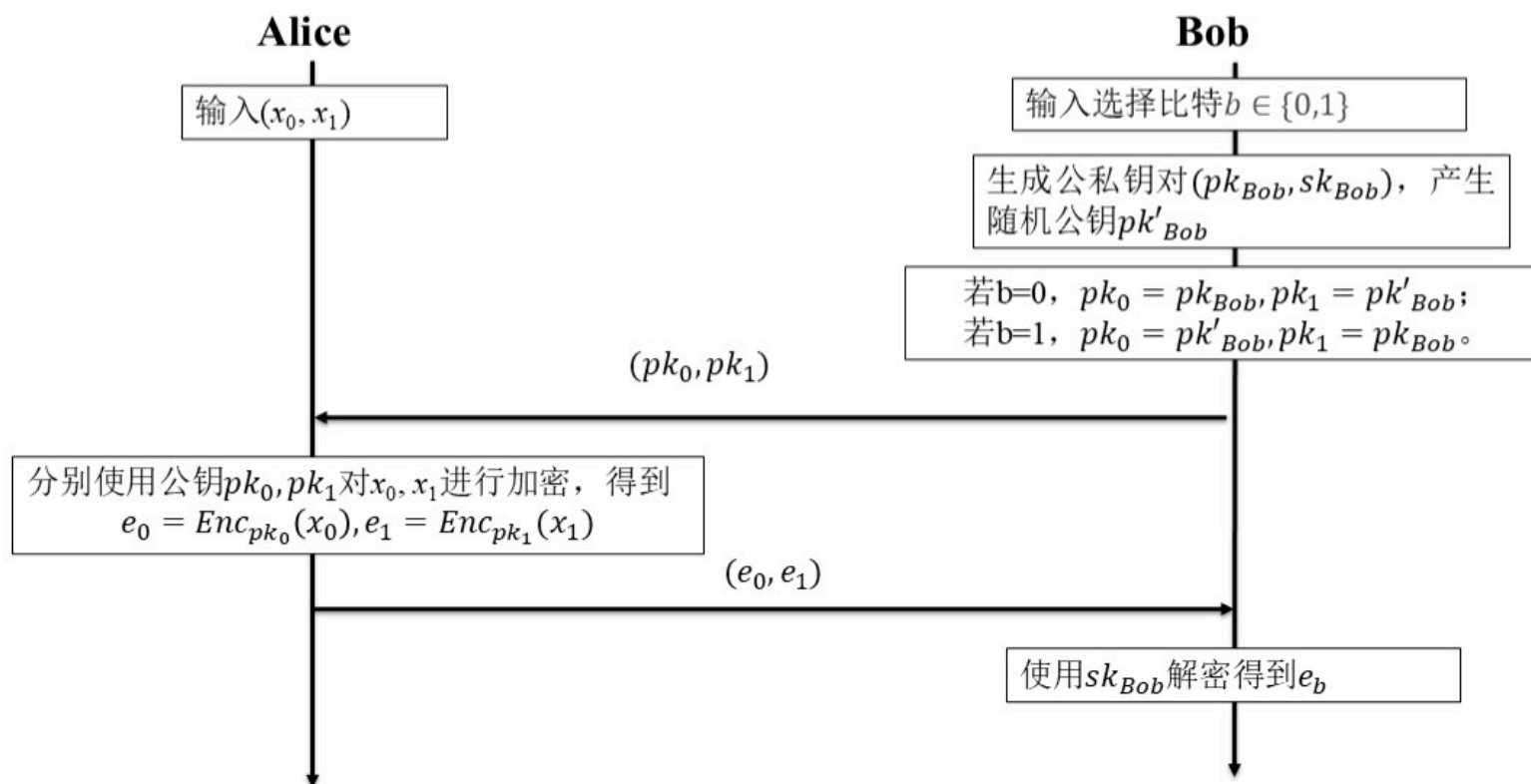
参数：

- 两个参与方：发送方 S 和接收方 R 。 S 的输入为秘密值 $x_0, x_1 \in \{0, 1\}^n$ ， R 的输入为选择比特 $b \in \{0, 1\}$ 。

协议：

1. R 生成一个公私钥对 (sk, pk) ，并从公钥空间中采样得到一个随机公钥 pk' 。如果 $b = 0$ ， R 将 (pk, pk') 发送给 S 。否则（如果 $b = 1$ ）， R 将 (pk', pk) 发送给 S 。
2. S 接收 (pk_0, pk_1) 并向 R 发送两个密文 $(e_0, e_1) = (Enc_{pk_0}(x_0), Enc_{pk_1}(x_1))$ 。
3. R 接收 (e_0, e_1) ，并用 sk 解密密文 e_b 。由于 R 不知道另一个密文所关联的密钥，因此 R 无法解密另一个密文。

2. 基础构造



2. 基础构造

回顾一下理想-现实范式。为了证明一个协议是安全的，在**理想世界中的攻击者必须能够生成一个视角，此视角与真实世界中的攻击者视角不可区分**，即：

- 攻击者能**在理想世界中生成一个真实世界中的“仿真”攻击者视角**，我们称此攻击者为仿真者，能说明存在这样一个仿真者，就能证明攻击者在现实世界中实现的所有攻击效果都可以在理想世界中实现；
- 如果**现实世界中攻陷参与方所拥有的视角和理想世界中攻击者所拥有的视角不可区分**，那么协议在半诚实攻击者的攻击下是安全的。

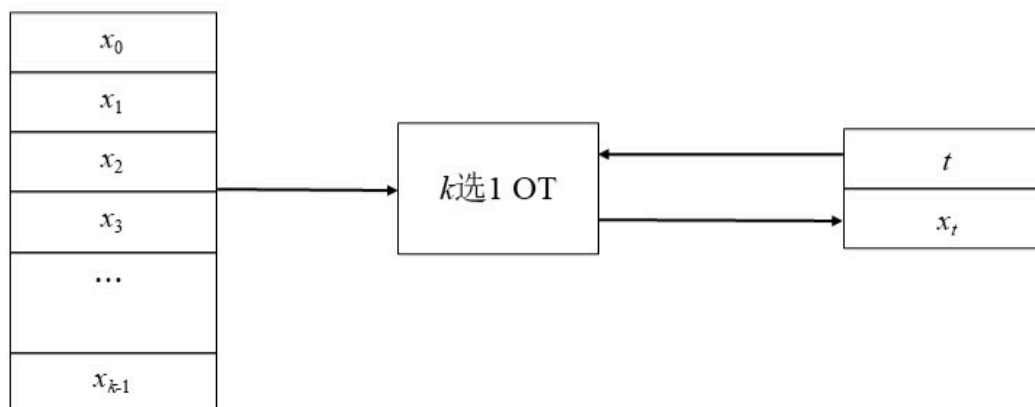
此协议在半诚实攻击模型下是安全的。发送方 S 只能看到由 R 发送的两个公钥，因此 S 无法以超过 $1/2$ 的概率预测出 R 拥有哪个公钥所对应的私钥。因此，仿真者可以直接向 S 发送两个随机选择的公钥，从而仿真出 S 的视角。

3. 基于公钥的k选1-OT协议

常见的“2选1”不经意传输常被扩展为“ k 选1”不经意传输。

如图所示，发送方S此时拥有 k 条隐私数据，接收方R拥有隐私的索引 $t \in \{0, 1, \dots, k-1\}$ 。在协议结束后，接收方R得到 x_t ，与“2选1”不经意传输一样，发送方S无法得知接收方R的隐私索引，接收方R也无法得到发送方S的其他数据。

□ 容易构造“ k 选1”不经意传输：将协议中生成1个随机公钥改为生成 $k-1$ 个随机公钥。



问题

□ 性能极差

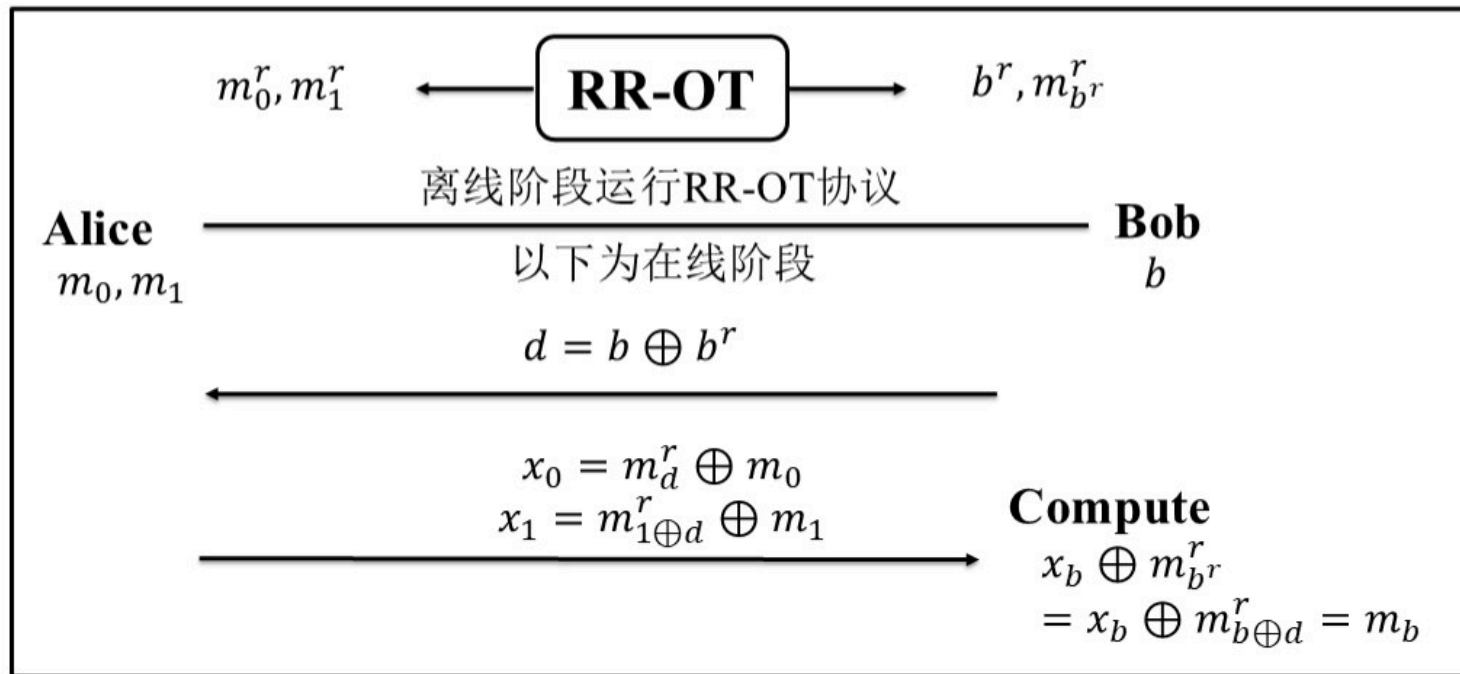
上述基础OT协议中，发送方和接收方对每一个选择比特都要执行系列公钥密码学操作

第一，可否利用对称密码来减少公钥密码学操作数量？

第二，是否可以将 n 位长度的数据用更短的OT来完成传递？

第三，可否利用预计算来提升在线时间的计算效率？

4. 预计算OT



- **离线阶段**，运行RR-OT协议（接收者随机的茫然传输），直接发送给发送方两个随机数，给接收方一个选择比特对应的随机数
- **在线阶段**，只需要通过异或运算就可以完成一个字符串的茫然传输

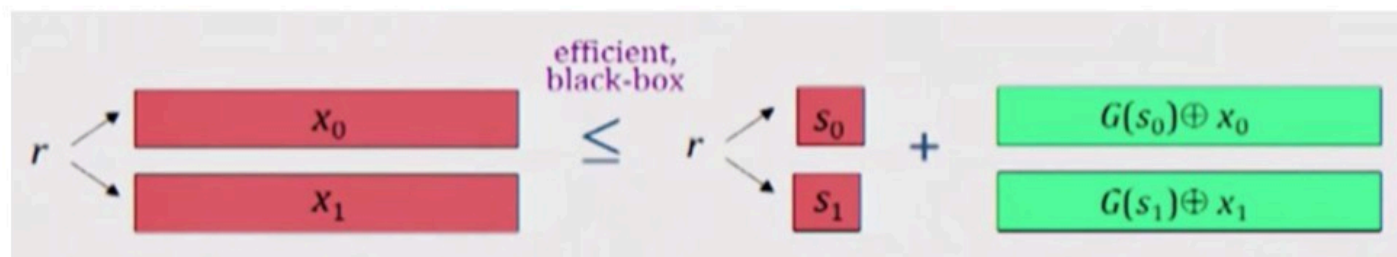
单选题 1分

有关茫然传输协议，说法错误的是

- ☐ A 基于公钥密码体制的2选1茫然传输协议性能较低
- ☐ B 在茫然传输协议中，接收方只能获取选择比特对应的数据
- ☐ C 在茫然传输协议中，发送方不能得知接收方选择了哪个数据
- ☒ D 茫然传输协议，只能通过公钥密码体制才可以完成构造

5. OT扩展

假设我们有一个可以传输短字符串的OT，通过使用标准的伪随机数生成器，我们能得到可以传输长字符串的OT。这一技术称为“**OT长度扩展**”。



□ 发送方将两个消息分别异或 $G(s_0)$ 或者 $G(s_1)$ ，将两个消息发送给接收方

□ 接收方对两个种子 s_0 和 s_1 用一个基础OT获得即可

使用短种子，利用伪随机函数放大变长

5. OT扩展

Ishai等人在2003年提出的OT扩展IKNP协议。此协议可在半诚实攻击者的攻击下实现 m 个2选1 OT协议，用来安全地传输 m 个随机字符串。

IKNP是第一个高效的 λ 个基础OT协议扩展为 $n(\gg \lambda)$ 个OT协议的工作。

因为通过一次茫然传输完成了多个OT示例的传递，因此称为**OT-实例扩展**

不经意传输扩展 (OT extension)

不经意传输扩展可以用于解决需要大量 OTs 的问题 [Beaver96, IKNP03]



- IKNP类 — PRG: [IKNP03, KK13, ALSZ13, ALSZ15, KOS15, OOS17, PSS17, Scholl18, Roy22], 利用伪随机生成器实现扩展
- PCG类 — LPN: [BCGI18, BCGIKS19, BCGIKRS19, SGRR19, YWLZX20, CRR21, GYWZXZL22], 基于LPN问题中噪音的稀疏性实现扩展
- PCF类 — LPN/DCR/DDH: [BCGIKS20, OSY21, BCGIKRS22, ADOS22], 目前实际效率低

不经意传输扩展 (OT extension)

大部分OT扩展协议:

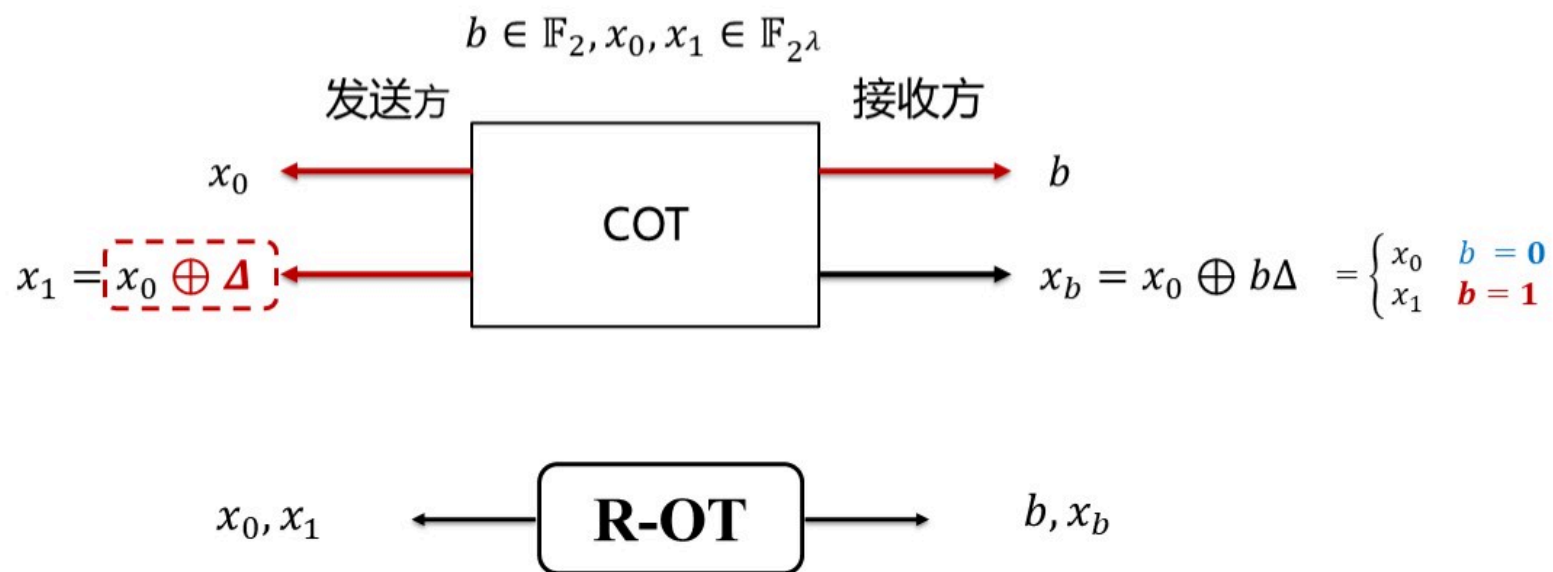
COT



ROT



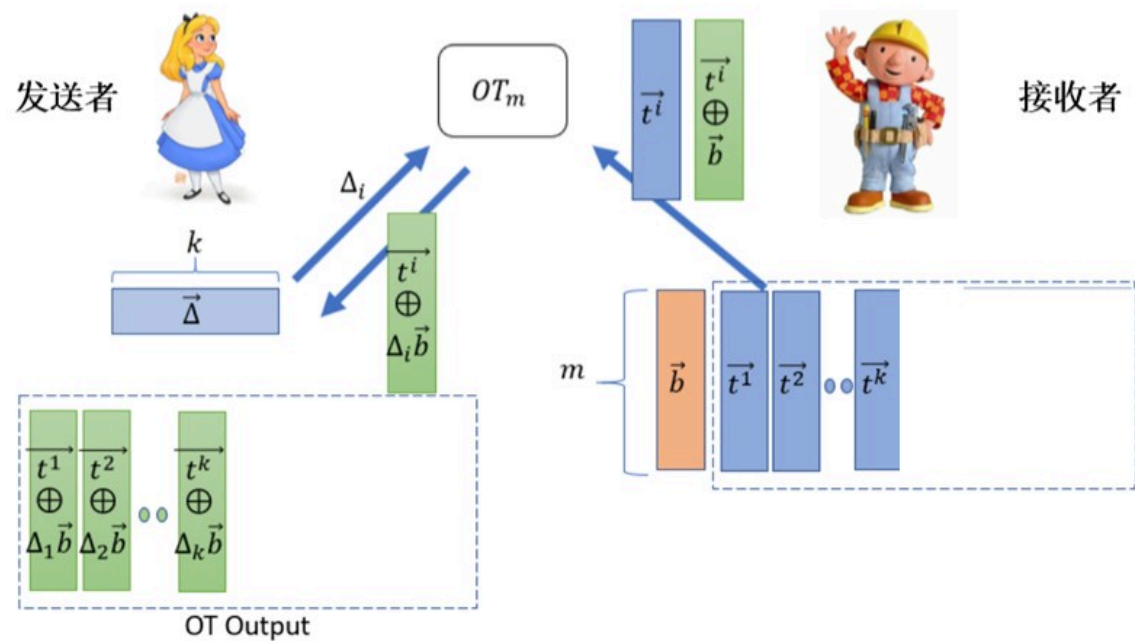
OT



IKNP类

1. 用公钥方法做固定数量 k （安全参数）次 **base OT**，每个OT随机串长度为 m
2. 利用矩阵转置，得到 m 个COT
3. m 个COT生成 m 个OT

步骤三：发送者和接收者准备一个 m 行 k 列矩阵以及选择向量 $\vec{b}$ ，发送者准备向量 $\vec{\Delta}$ 。对接收者的矩阵列执行 **base OT**，发送者要么得到 $\vec{t}^i$ ，要么得到 $\vec{t}^i \oplus \vec{b}$ 。



谢 谢

